

TEKNİK KAYNAK • HIL DOĞRULAMA SİSTEMİ

STM32 BLDC Kontrolü ve UART HIL Haberleşmesi

NUCLEO-G491RE üzerinde çalışan six-step komütasyon ve akım PI kontrolünün, PC'deki motor modeli ve masaüstü UI ile kapalı çevrim çalışması

STM32G491

115200 · 8-N-1

CRC16-CCITT

100 µs kontrol

20 kHz PWM modeli

PyQt6 UI

Hazırlanma tarihi

9 Eylül 2026

Odak

STM32 yazılımı, UART protokolü, PI ve komütasyon

Proje

blde_simula_stm + blde_desktop

Doğrulan senaryo

İleri → Coast → Reverse → Brake

1. Kısa cevap: burada ne yaptık?

Gerçek STM32 üzerinde çalışan bir BLDC kontrol algoritmasını, gerçek güç katı ve motor yerine PC'de çalışan ayrıntılı bir motor modeliyle kapalı çevrime aldık. STM32 hâlâ kontrol kararını veren taraftır: Hall bilgisini çözer, hangi MOSFET çiftinin sürüleceğini seçer, faz akımını ölçüm kabul eder ve PI ile duty üretir. PC ise bu komutun motor üzerindeki fiziksel sonucunu hesaplayıp Hall, enkoder, RPM ve akımları STM32'ye geri yollar.

4,0 A

varsayılan tork-akımı referansı

115200

LPUART1 baud, 8-N-1

100 μ s

bir kontrol/model çevrimi

PASS

ileri/coast/reverse/brake testi

En önemli sınır: Bu düzende STM32 gerçek, motor ve inverterin fiziksel davranışı simülasyondur. Böylece kontrol mantığı, komütasyon tablosu, PI tepkisi ve haberleşme güvenle doğrulanır; gerçek gate-driver zamanlaması, ADC gürültüsü ve güç elektroniği korumaları ise ayrıca gerçek donanımda test edilmelidir.

Test edilen varsayılan senaryo

UI şu anda **serbest rotor** senaryosunu çalıştırır. Modelde 24 V DC bara, 20 kHz anahtarlama PWM, 1 μ s iç çözüm adımı ve 0,01 Nm + $8 \times 10^{-7} \cdot \omega^2$ fan tipi yük vardır. Başlangıç akım referansı 4 A'dır. Bu, önceki "kilitli rotor/sadece akım PI" testinden farklıdır: rotor artık hızlanabilir, boşta yavaşlayabilir, ters torkla yön değiştirebilir ve dinamik frenlenebilir.

Akım $\neq$ hız: UI'deki 4 A alanı bir RPM hedefi değildir. Akım yaklaşık olarak torku belirler. Daha yüksek akım daha güçlü ivmelenme veya ters tork üretir; son hız, yük ile motor torkunun dengelendiği noktada oluşur. Sabit bir RPM hedefi istenirse akım PI'nin dışına ayrıca bir **hız PI döngüsü** eklenmelidir.

2. Sistem mimarisi

NUCLEO-G491RE / STM32

Gerçek çalışan kontrol yazılımı

- Hall $\rightarrow$ six-step komütasyon
- Akım PI + anti-windup
- Enable / yön / coast / brake
- CMD üretimi, FB ve CTRL ayrıştırma



PC / bldc_desktop

Motor fiziği + kullanıcı arayüzü

- 24 V inverter ve BLDC modeli
- Hall, enkoder, RPM ve akımlar
- Grafikler ve UART terminali
- Start / Coast / Brake / FWD / REV

CMD · 0x10 · STM32 $\rightarrow$ PC — mod, gate maskesi, A/B/C duty

FB · 0x11 · PC $\rightarrow$ STM32 — Hall, enkoder, RPM, faz akımları, Vbus, back-EMF, tork

CTRL · 0x13 · UI $\rightarrow$ STM32 — akım referansı, enable, yön, PI reset, brake

STM32'de kalanlar

- PI denklemi ve integral durumu
- Hall doğrulama ve sektör seçimi
- CW/CCW gate tablosu
- Coast/brake/six-step kararları
- CRC kontrolü ve link sayaçları

PC'ye taşınanlar

- Faz elektrik denklemleri
- Back-EMF ve elektromanyetik tork
- Rotor atalet/yük/sürtünme dinamiği
- PWM anahtarlama çözümü
- Sanal sensör geri beslemeleri

Bir HIL çevrimi nasıl ilerliyor?

- 1** STM32 ilk COAST CMD paketini yollar. Bu paket hem güvenli başlangıç komutudur hem de PC modeliyle el sıkışmayı başlatır.
- 2** PC'deki HilWorker CMD paketini CRC ile doğrular. Mod, gate maskesi ve duty değerini motor modeline uygular.
- 3** Motor modeli 100 μ s ilerletilir. İçeride 1 μ s'lik adımlarla 20 kHz PWM anahtarlama çözülür.
- 4** PC bir FB paketi üretir. Hall, enkoder, RPM, faz akımları, DC bara, yüzen faz gerilimi ve tork STM32'ye gönderilir.
- 5** STM32 FB'yi işler. Hall'dan aktif faz çifti bulunur, pozitif sürülen fazın akımı seçilir ve PI yeni duty'yi hesaplar.
- 6** Yeni CMD PC'ye döner. Böylece CMD → model → FB → PI → CMD kapalı çevrimi devam eder.

3. STM32 tarafında yaptıklarımız

3.1 Başlatma sırasını güvenli hale getirdik

`main.c` içinde önce iki UART kuruluyor, sonra HIL uygulaması ve PI hazırlanıyor. Kritik nokta, ilk CMD gönderilmeden **önce** kesmeli RX'in devreye alınmasıdır. Aksi halde PC'nin hızlı FB cevabı kaçabilir ve sistem bağlanır bağlanmaz susabilir.

```
MX_LPUART1_UART_Init();
MX_USART1_UART_Init();
HilApp_Init();
HilApp_SetCurrentRef(4.0f);
HAL_UART_Receive_IT(&hlpuart1, &s_hil_rx_byte, 1);
HilApp_Start(); /* RX hazırken ilk güvenli COAST CMD */
```

PC tarafı ilk komutu kaçırsa veya yeniden açılırsa, firmware 200 ms boyunca FB görmediğinde `HilApp_Start()` ile el sıkışmayı tekrar dener. Ana döngü `__WFI()` ile bekler; linkin yüksek frekanslı işi UART kesmesinde ilerler.

3.2 UART alımını kesme tabanlı ve kendi kendini toparlayan yaptık

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == LPUART1) {
        HilApp_OnHilRxByte(s_hil_rx_byte); // parser'a tek byte
        HAL_UART_Receive_IT(&hlpuart1, &s_hil_rx_byte, 1); // yeniden kur
    }
}
```

Her byte geldiğinde parser beslenir ve bir sonraki byte için RX hemen yeniden kurulur. Overrun/parite/hat hatasında HAL alımı durdurabildiğinden `HAL_UART_ErrorCallback()` içinde receive abort edilir ve kesmeli RX yeniden başlatılır. Bu ekleme, UI kapanıp açıldıktan sonra bağlantının kalıcı olarak sessizleşmesini önler.

3.3 İki UART'ın görevini ayırdık

Hat	Pin / fiziksel yol	İçerik	Gerekli mi?
LPUART1	PA2 TX / PA3 RX ST-LINK Virtual COM Port	CRC'li binary HIL: CMD, FB, CTRL	Evet. UI bu portu açar.
USART1	PC4 TX / PC5 RX harici USB-UART adaptörü	İnsan okunabilir DATA t=... debug satırı	Hayır. Bağımsız teşhis içindir.

Neden UI'de USART1 verisi görünüyör gibi? UI, binary FB paketindeki model verisini kendisi aynı DATA t=... biçimine çevirip mevcut grafik hattına verir. Yani UI terminalinde görülen DATA satırları için ikinci fiziksel UART kablosu gerekmez.

3.4 Non-blocking TX ve düşen paket sayaçları

`HilLink_Uart_Write()`, LPUART1 üzerinden `HAL_UART_Transmit_IT()` kullanır. UART meşgulse bloklamak yerine `g_hil_drops` artırılır. Benzer sayaç debug telemetrisi için `g_tel_drops`, RX hataları için `g_hil_rx_errors` olarak tutulur. Parser ayrıca `frames_ok` ve `crc_errors` sayar.

3.5 CTRL komutlarını firmware'e ekledik

Başta link yalnız STM32'den gelen sürüş komutu ve PC'den gelen sensör geri beslemesinden oluşuyordu. UI'den gerçek çalışma komutları verebilmek için 0x13 CTRL paketi eklendi. STM32 şu alanları uygular:

Flag	Değer	STM32 etkisi
ENABLE	0x01	Akım PI ve six-step sürüşü etkin.
REVERSE	0x02	Ters komütasyon tablosu seçilir.
RESET_PI	0x04	İntegral sıfırlanır; mod/yön geçişinde duty sıçraması önlenir.
BRAKE	0x08	Sürüş kapalıyken model HIL_MODE_BRAKE alır.

iref_mA firmware'de ampere çevrilir ve 0–10 A aralığında sınırlandırılır. Enable, brake veya yön değişirse integral otomatik sıfırlanır.

4. UART protokolü

4.1 Ortak çerçeve

A5 SOF0 1 byte	5A SOF1 1 byte	TYPE 1 byte	LEN 1 byte	PAYLOAD LEN byte · little-endian · padding yok	CRC LO 1 byte	CRC HI 1 byte
----------------------	----------------------	----------------	---------------	---	------------------	------------------

CRC16-CCITT kullanılır: polinom 0x1021, başlangıç 0xFFFF. CRC'ye SOF byte'ları katılmaz; yalnız TYPE + LEN + PAYLOAD üzerinden hesaplanır. CRC iki byte olarak little-endian, önce düşük byte gönderilir.

4.2 Paket özeti

Tip	Yön	Payload	Toplam	Amaç
CMD 0x10	STM32 → PC	12 byte	18 byte	İnverter modu, gate maskesi, üç duty
FB 0x11	PC → STM32	32 byte	38 byte	Sanal motor/sensör geri beslemesi
CFG 0x12	STM32 → PC	8 byte	14 byte	Gelecek yük/Vbus ayarı; şu an opsiyonel
CTRL 0x13	UI/PC → STM32	8 byte	14 byte	Çalışma, yön, akım ve fren komutu

4.3 CMD — STM32'nin inverter kararı

Ofset	Alan	Tip	Anlam
0	seq	u32	Komut sıra numarası
4	mode	u8	0 Coast, 1 Six-step, 2 senkron, 3 3-duty, 4 Brake
5	gates	u8	AH/AL/BH/BL/CH/CL bit maskesi
6, 8, 10	duty_a/b/c	u16	0...10000 = %0...%100

4.4 FB — PC modelinin sensör cevabı

Ofset	Alan	Tip / ölçek	Anlam
0	seq	u32	İşlenen CMD sıra numarası
4	t_us	u32, µs	Motor modelinin sanal zamanı
8	enc	i32	Sarmalanmamış x4 enkoder sayacı
12	speed_x10	i32, 0,1 rpm	Mekanik hız
16–17	hall, flags	u8 + u8	Hall bitleri ve enkoder index
18–23	ia, ib, ic	3x i16, mA	Faz akımları
24	vbus	u16, mV	DC bara
26	vfloat	i16, mV	Yüzen faz terminal gerilimi / back-EMF göstergesi
28	torque	i32, µNm	Model elektromanyetik torku

4.5 CTRL — kullanıcı komutu

Ofset	Alan	Tip	Anlam
0	seq	u32	UI kontrol sıra numarası
4	iref_mA	u16	0...10000 mA tork-akımı referansı
6	flags	u8	Enable / Reverse / Reset PI / Brake
7	reserved	u8	Gelecek kullanım; bugün 0

4.6 Byte-byte parser neden gerekli?

UART paket sınırı taşımaz; işletim sistemi veya kesme bir çerçeveyi parçalı verebilir. STM32 parser'ı S0F0 → S0F1 → TYPE → LEN → PAYLOAD → CRC0 → CRC1 durum makinesidir. Gürültüde A5 5A yeniden aranır, 64 byte'tan büyük LEN reddedilir ve CRC yanlışsa paket uygulanmaz. Böylece yarım, kaymış veya bozulmuş bir paket gate komutu üretemez.

5. Akım PI ve six-step komütasyon

5.1 PI denklemleri

$$e[k] = I_{ref} - I_{ölçülen}$$

$$I[k] = I[k-1] + K_i \cdot e[k] \cdot T_s$$

$$duty[k] = \text{clamp}(K_p \cdot e[k] + I[k], 0, 0,95)$$

Parametre	Değer	Not
K _p	0,1257	Anlık akım hatasına oransal tepki
K _i	146,6	Kalıcı hatayı integral ile giderir
T _s	100 µs	Her geçerli FB sonrasında bir sanal kontrol adımı
Duty sınırı	0...0,95	Modelde %95 üst sınır
Varsayılan I _{ref}	4,0 A	UI ile 0...10 A değiştirilebilir

Çıkış sınıra dayanırsa integral, taşan miktar kadar geri alınır. Bu back-calculation tipi anti-windup davranışı, özellikle yön değişiminde ve yüksek back-EMF altında integralin gereksiz büyümesini engeller. Ayrıca mod/yön değişiminde PI sıfırlanır.

5.2 Hall → sektör → gate seçimi

Hall kodu **Ha<<2** | **Hb<<1** | **Hc** olarak 1...6 aralığındadır. 000 ve 111 geçersiz kabul edilir. İleri yönde kullanılan tablo:

Hall	Sektör	High	Low	Float	Gate maskesi
001 / 1	1	A	B	C	AH + BL
010 / 2	5	C	A	B	CH + AL
011 / 3	6	C	B	A	CH + BL
100 / 4	3	B	C	A	BH + CL
101 / 5	2	A	C	B	AH + CL
110 / 6	4	B	A	C	BH + AL

Ters yönde **k_ccw** tablosu seçilir ve aynı Hall konumunda zıt tork üreten faz çifti uygulanır. Akım PI için ölçüm, gate maskesinde hangi high-side açıksa o fazın akımıdır.

5.3 Coast, Brake ve yön deęiřtirme

Coast

Tüm MOSFET'ler kapatılır. Elektromanyetik tork kesilir; rotor yük ve sürtünmeyle yavaşlar. Yavaşlama görece uzundur.

Brake

Modelde tüm low-side anahtarlar etkin dinamik fren durumuna geçer. Enerji elektriksel olarak sönümlenir; hız daha hızlı sıfıra yaklaşır.

Forward

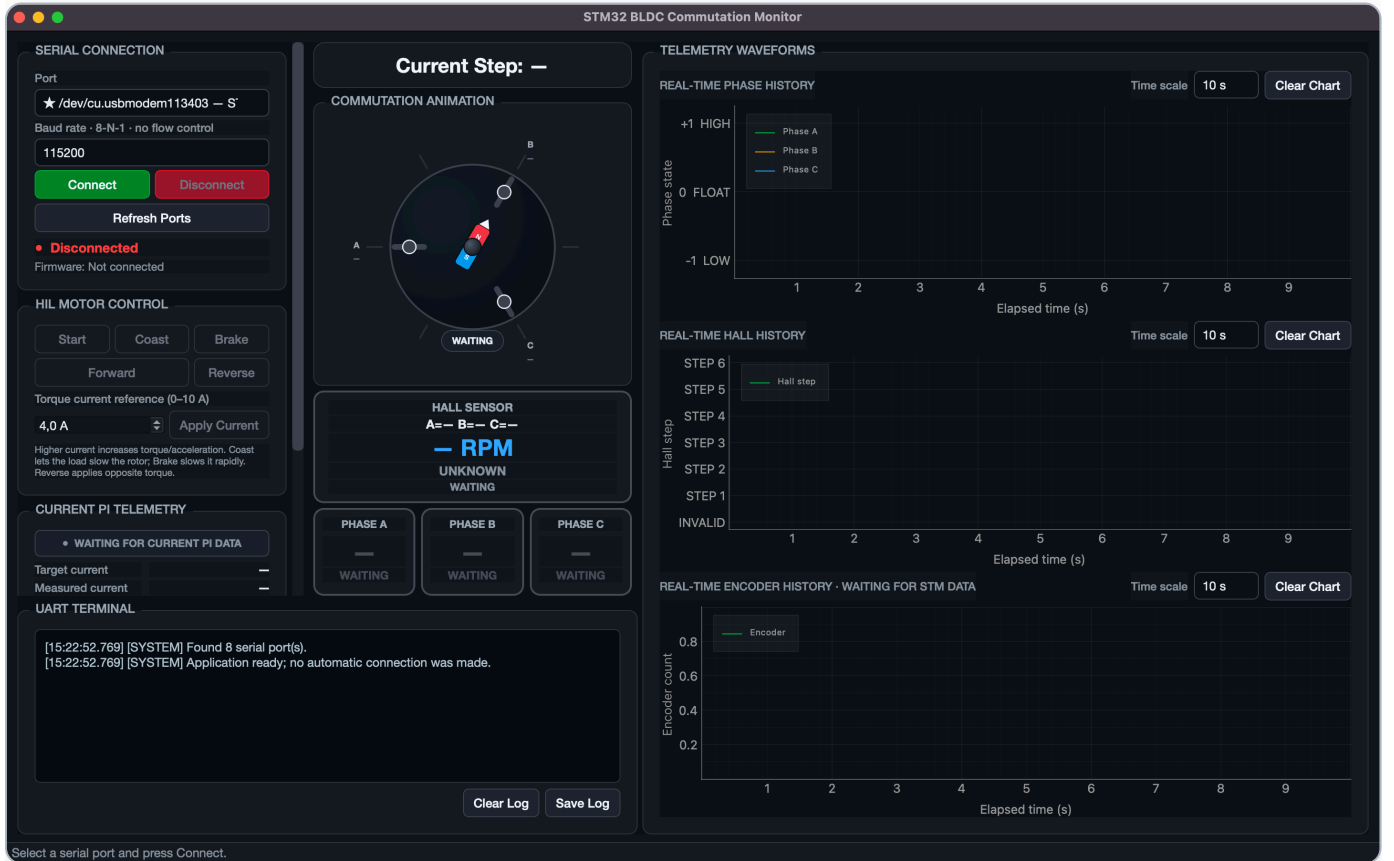
Enable açılır, CW tablo seçilir ve integral sıfırlanır. 4 A pozitif tork üretir.

Reverse

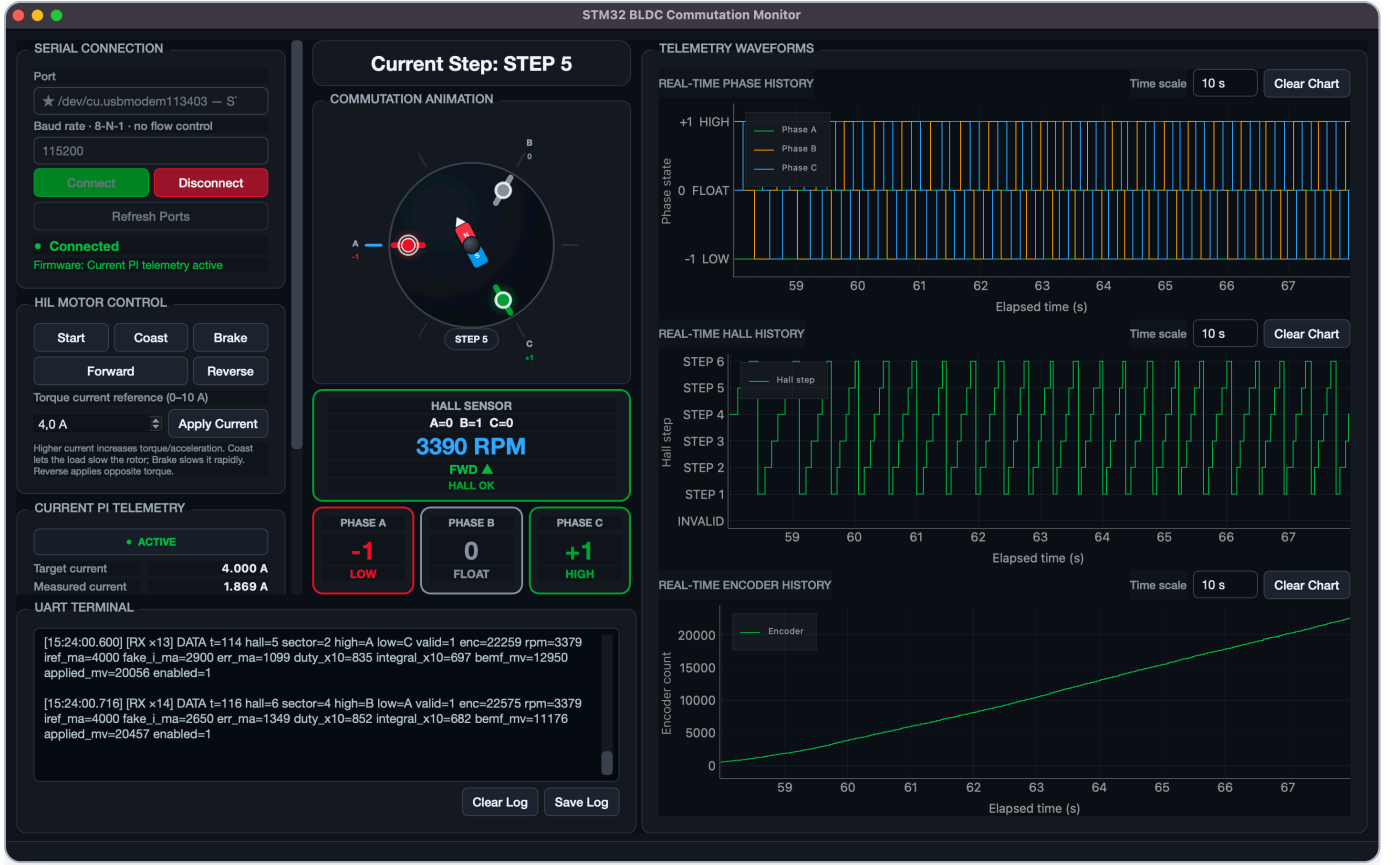
CCW tablo seçilir. Rotor ileri dönüyorsa önce ters torkla yavaşlar, sıfırı geçer ve negatif RPM'e hızlanır.

6. Masaüstü UI nasıl çalışıyor?

HilWorker seri portu ayrı bir Qt thread içinde açar. Böylece bağlantı, model hesabı ve UART beklemeleri arayüzü dondurmaz. Worker STM32'nin CMD'sini bekler, modeli ilerletir ve FB döndürür. Aynı örnekten `DATA t=...` satırı oluşturup mevcut telemetri parser'ına verir. UI düğmeleri ise komut kuyruğuna yazılır ve tek bir birleşik CTRL paketi olarak STM32'ye gönderilir.



Şekil 1 — Güncel UI, bağlantı öncesi. Sol sütunda yeni HIL Motor Control paneli; sağda faz, Hall ve enkoder grafikleri bulunur. Düğmeler bağlantı oluşana kadar güvenli biçimde pasiftir.

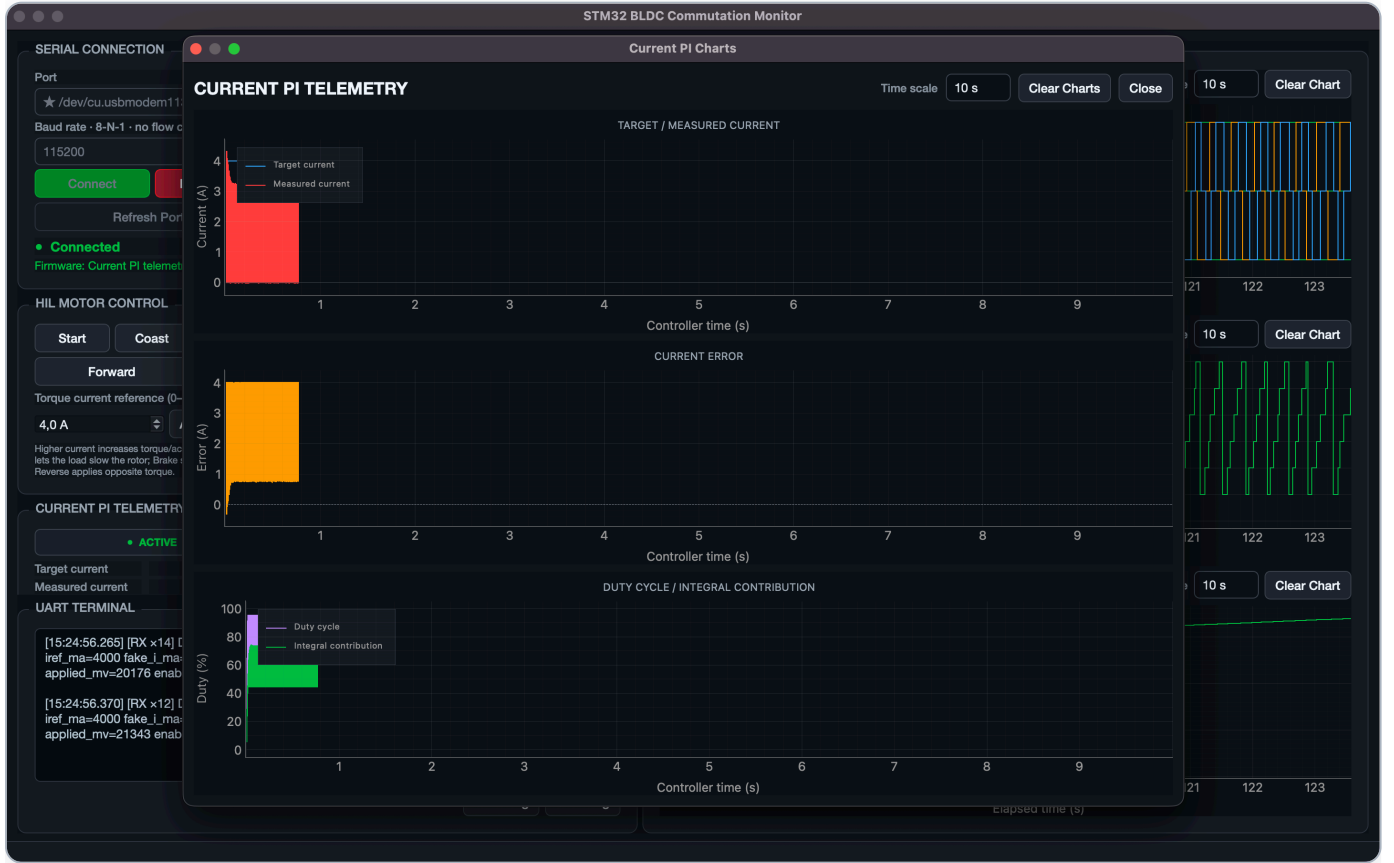


Şekil 2 — Gerçek STM32 bağlantısı çalışırken alınan ekran. Örnekte Hall=010, sektör 5, C high/A low, yaklaşık 3390 RPM ileri yön ve yükselen enkoder görülüyor. Terminalde modelden türetilen telemetri satırları akıyor.

Ekrandaki alanları okuma

Alan	Ne gösterir?	Sağlıklı işaret
Current Step	Hall kodundan çözölen six-step sektörü	Dönüşte 1...6 sıralı dolaşır
Phase History	Her faz: +1 high, 0 float, -1 low	Her anda bir high, bir low, bir float
Hall History	Komütasyon adım dizisi	Yöne göre düzenli ileri/geri sıra
Encoder	Sarmalanmamış konum sayacı	İleride artar, geride azalır
RPM	Modelin mekanik hızı	FWD pozitif, REV negatif
UART Terminal	Model + STM kontrol örneğinin özet satırı	valid=1, düzenli t/rpm/akım güncellemesi

7. PI grafiklerini doğru yorumlama



Şekil 3 — Serbest rotor, 4 A referans ve yüksek hız koşulunda PI grafikleri. Kontrolör zamanı sanal model zamanıdır; ana pencerenin elapsed eksenine aynı duvar saati değildir.

Üst grafik — Target / Measured Current

Mavi çizgi akım hedefi, kırmızı çizgi aktif high-side fazdan ölçülen akımdır. Kilitli rotor testinde kırmızı çizginin 4 A çevresine oturması beklenir. Serbest rotor ve anahtarlamalı six-step modelde komütasyon, PWM ripple'ı ve yükselen back-EMF nedeniyle dalgalanma normaldir. Ölçülen akım uzun süre hedeften düşük kalıyor ve duty %95'e dayanıyorsa motor hızında oluşan back-EMF nedeniyle 24 V bara artık hedef akımı üretmiyor demektir.

Orta grafik — Current Error

$\text{hata} = \text{hedef} - \text{ölçüm}$. Pozitif hata duty'nin artırılmasını, negatif hata azaltılmasını ister. Sıfır çevresinde küçük ve dengeli salınım iyi izlemeyi gösterir. Sürekli büyük pozitif hata, gerilim/duty sınırını veya yanlış aktif-faz ölçümünü; düzensiz 30–40 A sıçramalar ise zaman adımı/PWM örnekleme sorununu düşündürür.

Alt grafik — Duty / Integral Contribution

Mor çizgi STM32'nin CMD içindeki duty'sidir. Yeşil çizgi aynı FB örneği ve duty'den $I = \text{duty} - K_p \cdot e$ ile yeniden oluşturulan PI integral katkısıdır. Yüksek hızda back-EMF arttığı için aynı akımı korumak adına duty'nin yükselmesi doğaldır. Duty sürekli %95'teyse kontrolör saturasyondadır; akım hatası yalnız PI kazancı artırılarak çözülemez.

Önceki 0–40 A'lık “kırmızı duvar” neden oluşuyordu? PC modeli 100 μs adımla 20 kHz PWM'i çözmeye çalışırken iki tam PWM periyodu atlanıyor, sıfırdan büyük duty kimi adımlarda tam-bara ON gibi davranıyordu. Motor modeli iç adımı 1 μs 'e indirildi, PWM taşıyıcısı modulo ile sarıldı ve telemetri aynı FB/CMD örneğine hizalandı. Kilitli rotor doğrulamasında sonuç 4,003 A ortalama, 3,935–4,088 A aralığı oldu.

8. Doğrulanen hareket senaryosu

9 Eylül 2026'da fiziksel NUCLEO bağlantısı üzerinden otomatik test tekrar çalıştırıldı. Test sırayla FWD, COAST, REV ve BRAKE CTRL komutlarını gönderdi; 961 geçerli telemetri örneği toplandı, seri/model hatası oluşmadı.

Başlangıç 11 rpm	FWD sonrası 2847 rpm	COAST sonrası 1977 rpm	REV sonrası -2895 rpm	BRAKE sonrası -30 rpm
----------------------------	--------------------------------	----------------------------------	---------------------------------	---------------------------------

✓	İleri komutta hız büyüdü	11 → 2847 rpm
✓	Coast ile mutlak hız azaldı	2847 → 1977 rpm
✓	Reverse ile sıfır geçildi ve yön değişti	1977 → -2895 rpm
✓	Brake ile hız sıfıra yaklaştı	-2895 → -30 rpm
✓	Test sonucu	PASS · errors=[]

9. Build, flash ve çalıştırma

STM32 firmware

```
cd /Users/ynalbant/Desktop/STM32_Projects/bldc_simula_stm
make clean all
make probe
make flash
```

Çıktılar `Debug/bldc_simula_stm.elf`, `.hex` ve `.bin` olarak üretilir. Hedef kart NUCLEO-G491RE'dir. STM32CubeIDE kullanılıyorsa aynı proje açılıp Run ile de flash edilebilir.

UI

```
cd /Users/ynalbant/Desktop/bldc_desktop/motor_control_app
.venv/bin/python main.py
```

1. `/dev/cu.usbmodem...` ST-LINK portunu seçin.
2. Baud'u **115200** bırakın ve Connect'e basın.
3. Connected ve "Current PI telemetry active" mesajını bekleyin.
4. Akım referansını seçip Apply Current kullanın.
5. FWD/REV, Coast ve Brake davranışlarını RPM/enkoder grafiklerinden izleyin.

10. Sorun giderme

Belirti	Muhtemel neden	Kontrol / çözüm
Bağlanır bağlanmaz "Serial connection closed"	Worker kurulurken üçüncü konumsal argümanın yanlışlıkla <code>ts</code> olması veya worker istisnası	<code>HilWorker(port, baud, parent=self)</code> kullanılmalı. Hata callback'i UI terminaline gerçek exception'ı yazmalı.
Connected ama veri yok	İlk handshake kaçmış, RX kesmesi kurulmamış veya baud uyuşmuyor	115200 8-N-1; RX'i ilk CMD'den önce açın; 200 ms handshake retry aktif olmalı.
CRC hatası artıyor	Yanlış frame boyu, endian veya CRC kapsamı	CRC sadece TYPE+LEN+PAYLOAD; poly 0x1021/init 0xFFFF; CRC low byte önce.
Motor hızlanmıyor	Kilitli rotor senaryosu açık, enable kapalı, Iref=0 veya geçersiz Hall	<code>HIL_LOCK_ROTOR_CURRENT_TEST=False</code> ; START/FWD; 0'dan büyük Iref; <code>valid=1</code> .
Coast ile aniden durmuyor	Normal fizik	Coast torku keser; yük/sürtünme yavaşlatır. Hızlı duruş için Brake.
Reverse'e basınca hemen negatif olmuyor	Rotorun pozitif ataleti	Önce ters torkla yavaşlama, sonra sıfır geçişi ve negatif hız beklenir.
Akım hedef altında ve duty %95	Yüksek hız/back-EMF nedeniyle gerilim rezervi yok	Vbus/model parametreleri, hedef akım ve hız çalışma noktası incelenmeli.
UI'de terminal var ama USART1 kablosu yok	Bu beklenen tasarım	DATA satırını HilWorker üretir; HIL için yalnız ST-LINK VCP yeterlidir.

11. Sistem sınırları ve sonraki adımlar

Bu 10 kHz gerçek-zamanlı UART çevrimi değildir. CMD 18 byte, FB 38 byte'tır. 8-N-1'de bir alışveriş en az 560 bit taşır; 115200 baud'un teorik üst sınırı yaklaşık 205 çevrim/s'dir. Her çevrim modeli 100 µs ilerlettiği için sanal kontrol zamanı duvar saatinden yaklaşık 49 kat yavaş akabilir. Dolayısıyla bu sistem algoritma ve protokol HIL testidir; STM32 CPU yükü veya gerçek 10 kHz deadline testi değildir.

Üretime yaklaşmak için önerilen sıra:

1. UART HIL ile mevcut regresyonları otomatik çalıştırmaya devam edin.
2. Gerçek zaman gerekirse baud artırın, USB CDC/SPI/Ethernet veya paket gruplama yaklaşımı kullanın.
3. Gerçek inverterde dead-time, shoot-through koruması, over-current trip ve ADC tetikleme zamanını ekleyin.
4. RPM hedefi gerekiyorsa dış hız PI'si ekleyip çıkışını 0–10 A akım referansına sınırlayın.
5. Gerçek motorda önce düşük Vbus/akım limitiyle, osiloskop ve akım probu eşliğinde doğrulayın.

12. Kaynak kod haritası

Dosya	Sorumluluk
Core/Src/main.c	Saat/UART init, RX kesmesi, hata toplama, handshake retry, opsiyonel debug telemetrisi
Core/Src/hil_app.c	PI, kontrol durumu, FB/CTRL işleme, CMD üretme, DATA formatı
Core/Src/hil_link.c	CRC, byte parser, paket decode/build, Hall komütasyon tabloları
Core/Inc/hil_app.h	PI parametreleri ve HIL uygulama API'si
Core/Inc/hil_link.h	Protokol sabitleri, mod/gate/flag tanımları ve veri yapıları
bldc_desktop/.../services/hil_worker.py	Seri thread, lockstep model çevrimi, CTRL kuyruğu ve UI telemetrisi
bldc_desktop/.../protocol.py	Python frame/CRC/pack/unpack uygulaması
bldc_desktop/.../bldc_model.py	Elektriksel ve mekanik BLDC modeli, PWM, yük ve sensörler
tools/hil_motor_control_test.py	FWD → Coast → REV → Brake donanım-bağlantılı otomatik kabul testi

Özet zihinsel model

UI ne istendiğini söyler (CTRL) → STM32 nasıl süreceğine karar verir (CMD) → PC motorun ne yaptığını hesaplar (FB) → STM32 yeni kararı verir. Bu döngüde kontrolcü STM32'de, fiziksel plant PC'dedir.

Bu doküman, 9 Eylül 2026 tarihindeki doğrulanmış proje kaynakları ve canlı NUCLEO-G491RE HIL çalışması esas alınarak hazırlanmıştır. Sayısal sonuçlar sanal motor parametrelerine aittir; gerçek motor sonuçları olarak yorumlanmamalıdır.